

COMPREHENSIVE INTERVIEW PREPARATION

Security Hardware Design Verification Engineer

(Senior/Staff)

Qualcomm Cork, Ireland

GENERAL BEHAVIORAL & INTRODUCTION QUESTIONS

Q: Tell me about yourself

A: Concise, natural 60-second introduction covering: Name, background (Infosys 2y firmware, ISD 2y ASIC validation), MEng UCC (SERV thesis), Cork-based, 4+ years ASIC/embedded experience. Key strengths: verification rigor, systematic problem-solving, learning velocity. Position as senior-ready engineer excited about security IP.

Q: Why interested in Qualcomm Security HW DV role?

A: Cork location, security focus aligns with interests, AVA_VAN.5/EAL5+ standards demonstrate quality bar, role combines architecture collaboration + verification expertise. My SERV GLS work + AES UVM testbenches demonstrate readiness for security IP verification. Excited to contribute to products used in billions of devices.

Q: Where do you see yourself in 5 years?

A: Senior/Staff verification engineer or architectural verification lead. Deepen security IP expertise, lead complex verification initiatives, mentor junior engineers, contribute to Qualcomm methodologies, potentially technical leadership role. Make impact on how security is verified at scale.

Q: What are your biggest strengths?

A: Technical depth (understand micro-architecture deeply, not just verify), methodical problem-solving (break complex problems into modules), learning velocity (firmware→ASIC verification seamlessly), communication clarity. Directly applicable: can master cryptographic concepts, build comprehensive GLS environments, adapt to Qualcomm flows.

Q: Area you want to develop/improve?

A: Cryptographic algorithms and side-channel analysis are specializations. Proactively upskilled: studied AES deeply, researched S-boxes and Galois field math, exploring side-channel concepts. Acknowledge continuous learning domain; committed to closing gaps rapidly through hands-on work at Qualcomm.

Q: How do you handle stress/pressure?

A: Completed MEng while working full-time—demonstrated balance. Approach: prioritize ruthlessly, communicate blockers proactively, break large problems into milestones, take strategic breaks. GLS verification (tedious) maintained rigor through automation. Systematic approaches outperform heroics.

Q: Describe a failure and what you learned

A: Early Infosys: incorrect interrupt latency assumptions in FreeRTOS task scheduling. System failed hardware testing. Systematically measured actual latencies, discovered priority configuration issue. Lesson: measure before assuming, understand hardware/OS underneath. This is why I insist on architectural understanding in verification.

Q: Time you took leadership

A: At ISD, protocol validation framework fragile and hard to extend. Led redesign: analyzed pain points, designed new architecture (Python classes, data-driven config), automated test generators, migrated protocols incrementally. New framework accelerated onboarding 3x, reduced manual test writing 40%. Mentored next hire.

Q: Time you learned something quickly

A: MEng thesis required SystemVerilog RTL for bit-serial CPU. Minimal prior experience. Systematic approach: studied reference designs, built modules incrementally, wrote testbenches alongside RTL, engaged advisor regularly. Delivered thesis with original extensions, testbenches, GLS verification in 6 months.

Q: Complex technical problem you solved

A: GLS verification: simulation diverged between RTL and gate-level. Diagnosed via reduced test case, cycle-by-cycle waveform analysis, discovered SDF timing pessimistic on multiplexer chain. Worked with synthesis team to adjust constraints, reverified GLS. Emphasized importance of understanding SDF and delay propagation.

Q: Stay current with trends?

A: Follow DVcon/DAC, read whitepapers, take courses (advanced UVM), maintain knowledge base of patterns/anti-patterns, engage with peers. Stayed ahead on modern UVM, simulation acceleration, emerging verification trends.

SERV RISC-V PROCESSOR PROJECT - THESIS WORK

Q: SERV architecture and design

A: SERV is bit-serial RISC-V CPU processing 1 bit/cycle across 32 cycles. Extended with stochastic computing. Key modules: serv_top (integration), decode, ctrl (PC management), alu (bit-serial arithmetic), rf (register file dual-port), mem_if (load/store), state (state machine), csr (trap handling), bufreg/bufreg2 (shifters). Added stochastic elements to ALU for probabilistic computation.

Q: Advantages of SERV bit-serial

A: Area efficiency (1-bit datapath vs 32-bit), power efficiency (1 bit toggles per cycle), simplicity (smaller ALU/mux), scalability (easy to 64-bit), teaching value, reduced side-channel. Trade-off: latency (32 cycles/instruction). Informed verification strategy—couldn't use standard coverage models.

Q: Stochastic computing extension

A: Extended ALU to support stochastic operations: represent probabilities as bit streams (density encodes probability). Added control logic, reference model, testbenches with statistical checks. Characterized 25% power savings, <3% accuracy loss on neural network benchmarks. Thesis included synthesis, GLS, power characterization.

Q: SERV verification approach

A: Multi-level: unit tests (ALU, registers, decoder), module integration, full-system RTL, reference model, constrained-random + directed tests, coverage metrics (instruction, operand, state machine, control flow). Coverage goals guided testbench development.

Q: GLS verification process

A: Synthesized SERV with standard library, extracted SDF, ran identical testbenches at RTL and gate-level, instrumented with monitors, compared waveforms, debugged divergences. Discovered setup/hold violations on multiplexer chains, adjusted synthesis constraints, reverified. GLS pass with zero differences.

Q: Most challenging aspect

A: Stochastic verification—can't use exact output checks. Implemented statistical validation: tracked distribution statistics, used Kolmogorov-Smirnov tests to verify distributions matched expectations. Developed methodology for stochastic hardware reusable for other probabilistic designs.

Q: SERV security-focused redesign

A: Constant-time execution (eliminate data-dependent cycle counts), masked arithmetic, instruction cache with memory hiding, secure register file (EDC/ECC), fence instructions, interrupt masking during secrets, dynamic voltage/frequency scaling careful handling, formal verification for timing/power leakage. Would make SERV suitable for TEE.

Q: RISC_V_FORMAL interface usage

A: Added RVFI outputs for formal verification: rvfi_valid, rvfi_insn, rvfi_trap, rvfi_rs1/rs2_addr/rdata, rvfi_rd_addr/wdata, rvfi_pc_rdata/pc_wdata, rvfi_mem_*. Wrote SVA properties, used formal tool to verify register writes match semantics, memory addresses correct, PC updates for jumps/branches. Formal verification provided complementary assurance to simulation.

Q: Extend SERV experience to security IP

A: Apply architectural understanding to accelerators (AES, ECC, RSA). Study crypto algorithms deeply, implement reference models, verify functional properties (invertibility, commut), side-channel analysis (power/timing independence), fault injection (detect/mask), multi-block operations, integration testing, certification requirements (EAL5+).

AES-128 ENCRYPTION CORE PROJECT

Q: AES-128 architecture

A: Implemented 11 rounds (10 full + 1 final without MixColumns). S-box combinational logic (Boyar-Peralta optimized), state machine for round iteration, key schedule generating round keys on-the-fly, round operations (SubBytes, ShiftRows, MixColumns, AddRoundKey). 128-bit plaintext input, 128-bit ciphertext output, ~11 cycle latency.

Q: UVM testbench architecture

A: Interface (aes_if with clocking), Transaction (aes_item: plaintext, key, result, coverage), Driver, Monitor, Sequencer, Agent, Scoreboard (with reference model), functional coverage (pattern bins), test hierarchy (base/random/directed/corner-case), comprehensive reporting. Production-quality UVM environment.

Q: Constrained-random stimulus

A: Pattern selectors (RANDOM, ALL_ZERO, ALL_ONE, SAME_BYTE, ALT_A5/5A, BYTE_RAMP, WALKING_ONE/ZERO) with distribution bias (60% random, 4-6% structured). Independent constraints for plaintext and key. Coverage-driven generation—early tests broad, later focused on uncovered bins. Explores AES input space efficiently.

Q: AES reference model

A: Implemented FIPS 197 bit-exact in SystemVerilog: S-box lookup table, $GF(2^8)$ operations for MixColumns (XTime, field arithmetic), key schedule (SubWord, RotWord, Rcon), round functions. Verified byte-mapping matches RTL. Reference model ensures scoreboard checks valid.

Q: Functional coverage

A: Operand coverage (plaintext/key patterns—zero, ones, alternating, ramps, walking), cross-coverage (plaintext_pattern X key_pattern—36 bins), byte-level (ensure all byte positions exercised), round coverage (implicit via 11 rounds), output value binning, latency coverage, feature coverage (data_v_i, res_v_o, reset). Goals 95%+.

Q: AES side-channel verification

A: Power analysis: simulate AES with test vectors, capture toggling activity, analyze power traces for data dependence. If $\text{power}(\text{plaintext1}, \text{key}) \neq \text{power}(\text{plaintext2}, \text{key})$, key leaking. Timing: round latencies same for all inputs. DPA: correlation analysis between power traces and known

plaintext—should be ~zero. Fault injection: inject bit flips, verify detectably wrong outputs or masked. Formal: prove no data-dependent power leakage.

Q: S-box testing

A: Exhaustive testing all 256 inputs (trivial—only 256 cases). Verify each against known S-box table. Implementation-specific: verify intermediate signals correct (though not observable). Timing: purely combinational, no latency variation. Power: constant power regardless of input (resistance to power analysis). Formal verification: prove S-box matches FIPS spec bit-for-bit.

Q: Key Schedule verification

A: Known-vector testing (NIST vectors), algorithm verification (SubWord S-box ops, RotWord rotation, Rcon XOR), boundary testing (key bits), dependency analysis (each round depends on prior), timing (completes before first round or pipelined correctly), edge cases (zero/one keys), Rcon values correct. Formal: prove key schedule structure verifiable.

Q: MixColumns testing

A: Reference model in software, sample extensively (2^{128} infeasible, but sample well). NIST vectors include MixColumns intermediates. $GF(2^8)$ arithmetic verification (XTime for all 256 inputs, field operations properties—associativity, identity, inverses). Linearity: $MixColumns(A \oplus B) = MixColumns(A) \oplus MixColumns(B)$. Invertibility. Corner cases. Timing/power constant.

Q: AES in different modes

A: ECB (same plaintext→same ciphertext), CBC (ciphertext depends on prior blocks), CTR (stream cipher mode—counter increments, XOR plaintext). Multi-block sequences verify state transitions. NIST test vectors for modes. Performance: throughput bits/cycle. Modes usually software; AES core verifies block-cipher correctness only.

RV32I PROCESSOR PROJECT

Q: RV32I architecture

A: 5-stage pipeline: Fetch (PC management, instruction mem), Decode (instruction decoding, RF reads, immediate extraction), Execute (ALU, branch evaluation, address calc), Memory (load/store), Writeback (RF writes, result mux). Full RV32I ISA (ADD, SUB, logic, shifts, LW/SW, branches, JAL/JALR). Branch prediction (optional), hazard detection, forwarding, stalls/flushes, exception handling.

Q: Testbench hierarchy

A: Unit testbenches: tb_ALU (operations, flags, operands), tb_RegisterFile (write/read, ports, reset), tb_ControlUnit (decoding, control signals, illegal detection), tb_InstructionFetch (PC increment, targets, program counter), tb_RV32I_Processor (instruction sequences, program loading, trace verification). Hierarchical ensures component correctness and integration.

Q: Hazard detection and forwarding

A: Data hazards (RAW): ADD R1, R2, R3 followed by ADD R4, R1, R5. Verify forwarding detects R1 dependency, forwards ALU result, OR stall inserted. Test all forwarding paths (EX→EX, MEM→EX, WB→EX). Control hazards: branch outcome not known until EX stage. Verify PC updated to target or sequential, branch prediction tested, incorrect instructions flushed.

Q: Load/store operations

A: Address calculation (base + immediate, ranges, boundary crossing). Load operations (LW/LH/LB with sign/zero extension). Store operations (SW/SH/SB). Alignment (aligned/misaligned). Memory interface (address/data timing). Hazard interactions (load forwarding, store address dependencies, back-to-back memory ops).

Q: Branch and jump instructions

A: Branch types (BEQ, BNE, BLT, BGE, BLTU, BGEU). Test conditions (taken, not taken, boundaries). Target calculation (positive/negative offsets, large offsets). Jump instructions (JAL saves return address, JALR flexible targets). Instruction sequences (dependent conditions, nested branches, mispredicts if prediction present). Traps (invalid address, misaligned targets).

TINYSOC INTEGRATION PROJECT

Q: TinySoC architecture

A: Integrates CPU (tiny_cpu), RAM (128x8), UART MMIO, Timer MMIO on simple bus (soc_bus). Bus arbitrates between CPU master and slave peripherals. Chip-select decoding. Testing: unit tests (RAM, UART, Timer), bus routing, integration end-to-end scenarios.

Q: UART testing

A: TX: write data, verify bit-serial output (start, 8 data, stop bits), baud rate timing, back-to-back characters. RX: input bit-serial, verify byte reconstruction, data available in register, error handling (framing, overrun). Handshaking (status flags, interrupts). FIFO (if present). Functional: echo program (CPU reads RX, writes TX).

Q: Timer interrupt testing

A: Load timer count, verify countdown, assert interrupt when zero, optional auto-reload. Interrupt handling: CPU recognizes interrupt, executes handler, clears flag, resumes. Timing accuracy (interrupt at expected cycle). Back-to-back interrupts. Context save/restore. Functional: periodic LED blinks via interrupts.

Q: Bus arbitration and chip-select

A: Address decoding (define map: RAM 0x00-0x7F, UART 0x80-0x8F, Timer 0x90-0x9F). Verify correct chip-select asserted, mutually exclusive, during valid transactions. Data routing (master issues address, correct slave captures, returns read data). Timing (address stable, data valid). Edge cases (address transitions, back-to-back, different slaves). Bus contention/arbitration.

Q: Test program for TinySoC

A: Initialize stack/timer/UART. RAM test (patterns, verify). UART echo (read RX, transmit TX).
Timer test (set interrupt count, wait for interrupts). Integration (write counter via UART).
Completion (test-passed signature on GPIO). Exercises all subsystems, demonstrates end-to-end functionality.